PROGRAMACIÓN CONCURRENTE · PROYECTO INTEGRADOR P4

PixelFlow

Pipeline concurrente de procesamiento de imágenes en Go: goroutines, canales acotados, fan-out/fan-in, `errgroup` y `context`.

SH02 · Aplicación concurrente Go — Informe técnico

1.95×
speedup medido con 2 núcleos (120 imágenes 800×600)

0
carreras con `go test -race -count=3` y 0 goroutines colgadas

87–99 %
cobertura de pruebas en todos los paquetes con lógica

INTEGRANTES

Francisco Rafael Corzo Pérez
Leonardo de Jesús Martínez Martínez

TECNOLOGÍA

Go 1.22+ (probado 1.24.7)
golang.org/x/sync/errgroup

FECHA

8 de noviembre de 2026

1 Resumen

Construimos **PixelFlow**, una herramienta de línea de comandos en Go que procesa carpetas de imágenes JPEG/PNG con un pipeline concurrente **listar → decodificar → filtrar → codificar**, con número configurable de goroutines por etapa, canales con búfer acotado, cancelación con `context` (Ctrl+C o plazo) y propagación del primer error con `errgroup`. Implementa escala de grises, desenfoque y redimensionado, más Sobel, paralelismo por franjas y una interfaz web como extensiones. Sobre 120 imágenes de 800×600 el tiempo baja de **4.44 s** (secuencial) a **2.28 s** con 2 goroutines por etapa: **speedup 1.95×** en un equipo de 2 vCPU, consistente con una fracción serial de Amdahl de **2.6 %**. Las salidas son idénticas byte a byte a la versión secuencial y la suite pasa `go test -race -count=3`.

1.95×
speedup con GOMAXPROCS = 2 (máx. ideal 2×)

2.6 %
fracción serial estimada (Amdahl) → tope 38.2×

41 + 24
pruebas + subpruebas; -race limpio ×3

48 ms
en detenerse tras Ctrl+C real (300 imágenes)

2 Arquitectura concurrente

El sistema es un pipeline de cuatro etapas conectadas por tres canales. Cada etapa, salvo el listado, es un **worker pool** de tamaño configurable (`-decoders`, `-workers`, `-encoders`): los workers de una etapa compiten por el mismo canal de entrada (**fan-out**) y escriben en un único canal de salida (**fan-in**). Todas las goroutines pertenecen a un mismo `errgroup`, de modo que `Run` nunca regresa antes de que todas hayan terminado. El pipeline no conoce el disco: lee y escribe a través de dos puertos (`Source`, `Sink`) y forma parte de una arquitectura en capas cuyas dependencias apuntan hacia el dominio (Anexo E).

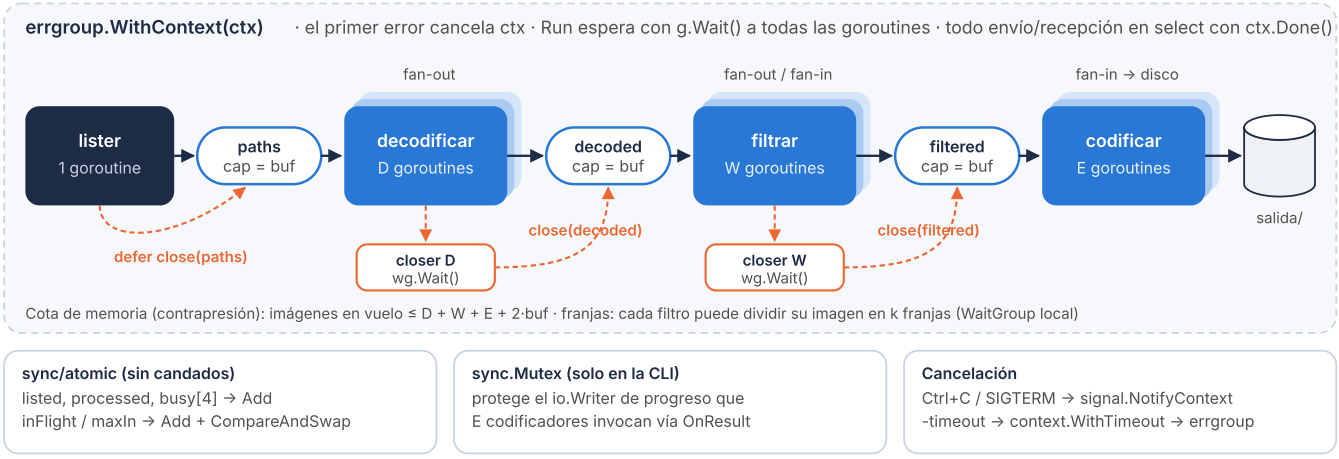


Figura 1. Goroutines, canales (con capacidad), responsable de cada `close` (naranja) y primitivas de sincronización. D, W, E y buf son flags de la CLI.

3 Propiedad y ciclo de vida

Canales

Canal (cap.)	Escribe	Lee	Cierra
`paths` (buf)	lister	D decodificadores	lister, `defer close`
`decoded` (buf)	D decodificadores	W filtros	closer D tras `wg.Wait()`
`filtered` (buf)	W filtros	E codificadores	closer W tras `wg.Wait()`

Los workers reciben canales **direccionales** (`in <-chan T`, `out chan<- U`): el compilador impide que un worker cierre su salida. Cada `close` aparece en un solo lugar, así que el pánico por cierre múltiple (riesgo típico de P4) no puede ocurrir.

Goroutines

Goroutine	La crea	Termina cuando…
lister ×1	`Run` vía `g.Go`	envió todo, falló `ReadDir` o `ctx` cancelado
decodificador ×D	`stage()` vía `g.Go`	`paths` cerrado y vacío, error o `ctx`
filtro ×W	`stage()` vía `g.Go`	`decoded` cerrado y vacío o `ctx`
codificador ×E	`stage()` vía `g.Go`	`filtered` cerrado y vacío, error o `ctx`
closer ×2	`stage()` vía `g.Go`	justo después del último worker de su etapa
franja ×k	`filters.forRows`	al terminar su franja (`WaitGroup` local)

4 Decisiones de diseño

Necesidad	Primitiva elegida	Por qué · alternativa descartada
Mover imágenes entre etapas	Canales con búfer acotado (`-buf`, def. 4)	Dan contrapresión gratis y se combinan con `select`. *Descartado:* slice + `Mutex` + `sync.Cond`: más código, no cancelable con `ctx`, y sin cota natural de memoria.
Cerrar la salida de una etapa con N productores	`sync.WaitGroup` + goroutine closer	Un único responsable que cierra solo cuando ya nadie puede enviar. *Descartado:* que cada worker cierre (pánico por cierre múltiple) o un contador compartido de «últimos» (frágil).
Primer error cancela todo	`errgroup.WithContext`	Registra el primer error, cancela el `ctx` compartido y `Wait` espera a todos. *Descartado:* canal de errores manual con `sync.Once`: reimplementa errgroup y es fácil dejar un emisor bloqueado.
Ctrl+C y plazo	`signal.NotifyContext`, `context.WithTimeout`	La señal se convierte en cancelación del mismo `ctx` raíz. *Descartado:* `os.Exit` en el manejador: deja archivos a medio escribir y no imprime el resumen parcial.
Contadores y máximo de imágenes en vuelo	`sync/atomic` (`Add`, `CompareAndSwap`)	Se actualizan en la ruta caliente desde todas las etapas. *Descartado:* `Mutex`: contención innecesaria para un entero.
Progreso en la CLI (`OnResult`)	`sync.Mutex`	El `io.Writer` no es seguro en concurrencia; **`-race` detectó esta carrera** en la primera versión (ver §5).
Entrada y salida de imágenes	Interfaces `Source` / `Sink` (puertos)	El pipeline se prueba en memoria y el disco es un adaptador; escritura atómica con temporal + `os.Rename`. *Descartado:* rutas fijas en `Config`, que acoplaban el motor al disco.

5 Corrección

Riesgo	Cómo se evita	Prueba que lo verifica
Carreras de datos	Los filtros son funciones puras (no mutan la entrada); cada imagen pertenece a una sola goroutine a la vez (se transfiere por canal); los contadores son atómicos; las franjas escriben filas disjuntas.	`go test -race -count=3 ./...` limpio; `TestStripsIdenticalToSequential`
Interbloqueo	El grafo de canales es acíclico; no hay candados entre etapas; todo envío/recepción bloqueante está en `select` con `ctx.Done()`; los canales se cierran tras `wg.Wait()`, por lo que todo `range`/recepción termina.	`TestConcurrentMatchesSequential` (24 combinaciones W×buf×franjas, incl. buf = 0)

Inanición	Los workers de una etapa son simétricos y consumen del mismo canal FIFO; ningún worker retiene un recurso compartido mientras espera.	`TestEachImageExactlyOnce` (64 goroutines, cada índice entregado exactamente 1 vez)
Fugas de goroutines	`Run` regresa solo tras `g.Wait()`; las franjas esperan con `WaitGroup`. Sin `time.Sleep` para sincronizar.	Helper `checkNoLeaks` (`runtime.NumGoroutine`) en todas las pruebas del pipeline
Primer error / cancelación	`errgroup` devuelve el primer error (no el `context.Canceled` derivado) y cancela al resto.	`TestFirstErrorCancelsAll`, `TestCancellationStopsQuickly` (< 1 s), `TestDeadlineAndPreCancelled`
Memoria sin cota	Imágenes en vuelo ≤ D + W + E + 2·buf, medido con `Stats.MaxInFlight`.	`TestBackpressureBound`, `TestEachImageExactlyOnce`
Filtros incorrectos	Aritmética entera con redondeo explícito; valores esperados calculados a mano.	Pruebas píxel a píxel de gris, desenfoque, resize y Sobel

Carrera real encontrada. La primera versión de la CLI imprimía el progreso desde el callback `OnResult`, que invocan los E codificadores a la vez. `go test -race` reportó *DATA RACE* en `TestCLI` (escrituras concurrentes al `bytes.Buffer`). Se corrigió serializando con un `sync.Mutex`; desde entonces la suite pasa limpia. Para la defensa, `go test -race -tags racedemo` reproduce a propósito una carrera sobre un contador `int`.

Demostración de Ctrl+C. Con 300 imágenes, enviar `SIGINT` produjo «Resumen: 46/300 imágenes», código de salida 130, detención en ≈48 ms, 46 archivos completos y **cero temporales**. Cobertura: pipeline, filters y bench 98.7 %, imgio 98.2 %, app 90.2 %, CLI 89.7 %, web 87.3 %. Los adaptadores en memoria prueban errores de lectura, decodificación y escritura sin tocar el disco.

6 Resultados experimentales

Equipo: VM Linux 6.18 ×86-64, Intel Xeon @ 2.30 GHz, **2 vCPU**, 7 GB RAM, Go 1.24.7. **Carga:** 120 imágenes sintéticas 800×600 (60 PNG, 60 JPEG, 28 MB), cadena `gray,blur:2,resize:50%`. **Método:** `scripts/bench.sh`, **5 repeticiones** por configuración, intercaladas; se reporta media ± desviación estándar. El speedup es $T_{secuencial}/T_P$, con P = GOMAXPROCS = goroutines por etapa. Además, `go test -bench -cpu 1,2,4,8 -count 5` sobre 100 imágenes 320×240 (Anexo B).

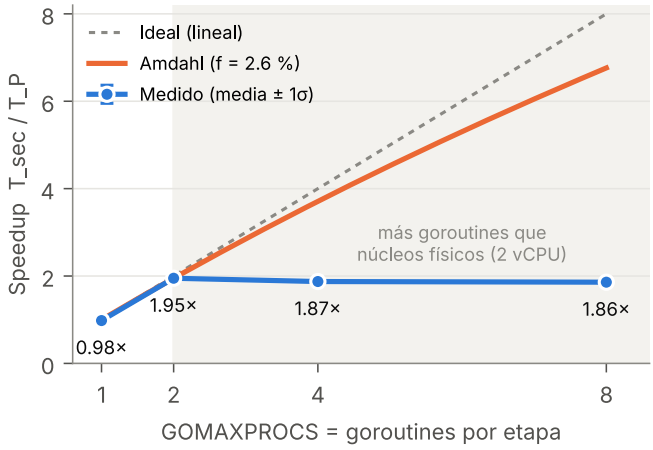


Figura 2. Speedup medido frente a Amdahl (ajustado con P = 2) e ideal. Barras: ±1σ.

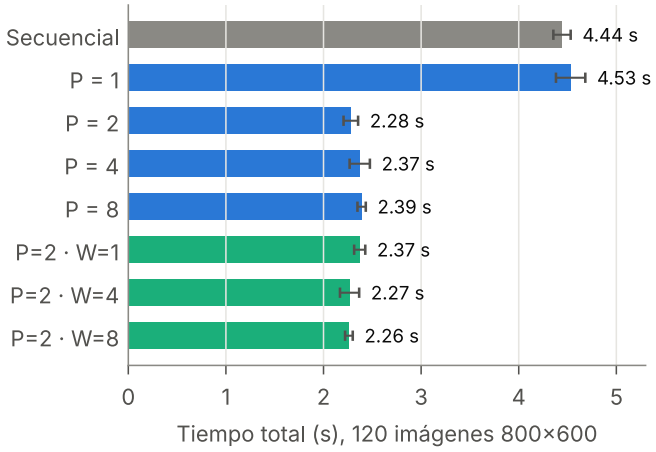


Figura 3. Tiempo por configuración (media ± 1σ, n = 5). Verde: GOMAXPROCS = 2 variando goroutines por etapa.

Interpretación. Con 2 goroutines por etapa el speedup es **1.95×** (eficiencia 97.5 %); la variabilidad es baja (CV ≤ 4.4 %). Con P = 1 el pipeline es 2.0 % más lento que la versión secuencial: es el costo de canales, cambios de contexto y contadores. De P = 2 a P = 8 el speedup cae levemente (1.87× y 1.86×) porque el equipo solo tiene **2 núcleos físicos**: más goroutines no agregan cómputo, solo planificación y presión sobre caché y GC. La serie verde muestra que incluso con **una goroutine por etapa** se obtiene 1.87×: las tres etapas ya corren en paralelo entre sí (paralelismo de pipeline), pero limitadas por la etapa más lenta.

Amdahl. Despejando S(2) = 1 / (f + (1−f)/2) se obtiene **f ≈ 2.6 %** de trabajo serial (listado, llenado y vaciado del pipeline, pausas de GC y sobrecosto de sincronización). Con esa f, Amdahl predice **3.71×** con 4 núcleos, **6.76×** con

8 y un máximo teórico de **38.2×**. Las predicciones para 4 y 8 no se pueden confirmar en este equipo; `scripts/bench.sh` está listo para repetirse en una máquina del laboratorio con 8 núcleos.

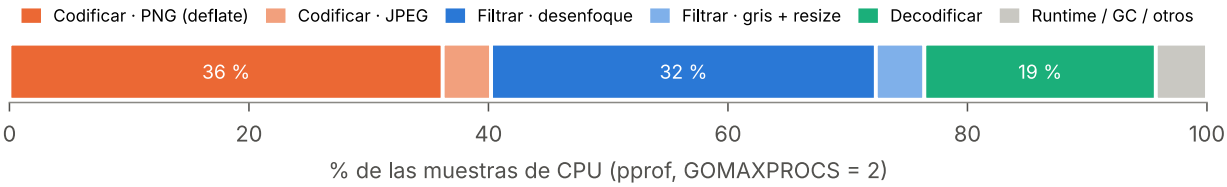


Figura 4. Perfil de CPU (`pprof`, 2.39 s de reloj y 4.45 s de CPU = 186 % de uso, es decir, ambos núcleos saturados). Detalle en el Anexo C.

Cuello de botella. La etapa más costosa es **codificar** (40 % de la CPU), dominada por la compresión deflate de PNG (`compress/flate.findMatch` es la función con más tiempo propio, 23 %). Le siguen el desenfoque (32 %) y la decodificación JPEG/PNG (19 %). Con los archivos en caché del sistema operativo todas las etapas resultan **limitadas por CPU**; la E/S real es despreciable frente a comprimir y descomprimir. Por eso el número de goroutines útiles está acotado por los núcleos y no por la latencia de disco, y por eso conviene dar a `-encoders` al menos tantas goroutines como a `-workers`. Si los canales intermedios tuvieran búfer de un millón, la contrapresión desaparecería: el decodificador llenaría la memoria con todas las imágenes (≈1.9 MB cada una) mientras el codificador, más lento, se queda atrás, sin ganar velocidad.

7 Limitaciones y trabajo futuro

- **Hardware de medición:** 2 vCPU impiden observar escalamiento real con 4 y 8; se debe repetir `bench.sh` en un equipo de ≥ 8 núcleos para validar la predicción de Amdahl.
- **Codificación PNG:** deflate es secuencial por imagen y domina el perfil; una flag `-compresion` (p. ej. `png.BestSpeed`) o `sync.Pool` para los `png.EncoderBuffer` reduciría la etapa crítica.
- **Redimensionado:** vecino más cercano (exacto y verificable píxel a píxel, con aliasing); bilineal o Lanczos serían el siguiente paso.
- **Alcance:** falta la extensión `-orden`; cada imagen se procesa completa en memoria; la interfaz web es local, sin autenticación, y admite un trabajo a la vez (ADR 0004). El historial de Git consolida el trabajo: cada integrante debe subir sus commits diarios.

8 Uso de herramientas de IA

Un asistente de IA (Claude, de Anthropic) generó la primera versión del código, las pruebas, la interfaz web, la reorganización en capas, los scripts y este informe a partir de la propuesta y la rúbrica. Se verificó ejecutándolo: `vet` / `gofmt` limpios, `go test -race -count=3` en verde, cobertura, Ctrl+C real, recorridos de la interfaz en un navegador automatizado y benchmarks repetidos tras la reorganización; las cifras salen de `results/`. La verificación halló y corrigió defectos reales (la carrera de §5 y respuestas JSON con `null`). El equipo revisó el código para explicarlo en la defensa.

Anexo A · Datos experimentales completos

CLI, 120 imágenes 800×600, `gray,blur:2,resize:50%`, 5 repeticiones. Fuente: `results/cli_runs.csv`.

Modo	GOMAXPROCS	D/W/E	n	Media (s)	σ (s)	CV	Mín–máx (s)	Speedup
Secuencial (referencia)	1	—	5	4.443	0.089	2.0 %	4.320–4.538	**1.00×**
Concurrente	1	1/1/1	5	4.532	0.150	3.3 %	4.421–4.785	**0.98×**
Concurrente	2	2/2/2	5	2.280	0.075	3.3 %	2.202–2.403	**1.95×**
Concurrente	4	4/4/4	5	2.371	0.104	4.4 %	2.246–2.509	**1.87×**
Concurrente	8	8/8/8	5	2.390	0.043	1.8 %	2.333–2.442	**1.86×**
Concurrente	2	1/1/1	5	2.371	0.057	2.4 %	2.328–2.469	**1.87×**
Concurrente	2	4/4/4	5	2.267	0.098	4.3 %	2.206–2.439	**1.96×**
Concurrente	2	8/8/8	5	2.260	0.039	1.7 %	2.223–2.326	**1.97×**

Anexo B · `go test -bench` con GOMAXPROCS = 1, 2, 4, 8

100 imágenes 320×240, `-count 5`, ms por ejecución completa (media ± σ). Fuente: `results/bench.txt`. Con -cpu 2 y 4 workers: **1.77×** frente al secuencial con -cpu 1.

Configuración	-cpu 1	-cpu 2	-cpu 4	-cpu 8
`secuencial`	597 ± 14	605 ± 14	628 ± 6	662 ± 29
`workers=1`	593 ± 21	358 ± 18	427 ± 28	456 ± 10
`workers=2`	618 ± 20	340 ± 6	389 ± 21	411 ± 17
`workers=4`	588 ± 10	338 ± 17	382 ± 13	392 ± 23
`workers=8`	582 ± 30	324 ± 14	370 ± 13	374 ± 20

Con imágenes pequeñas la parte fija (listar, crear goroutines, abrir archivos) pesa más, por eso el speedup es menor que en la CLI. El patrón es el mismo: la mejora se concentra entre 1 y 2 núcleos y luego se estabiliza.

Anexo C · Perfil de CPU (`go tool pprof -top`)

```
Showing nodes accounting for 4.18s, 93.93% of 4.45s total
Dropped 86 nodes (cum <= 0.02s)
Showing top 30 nodes out of 84
      flat  flat%   sum%        cum   cum%
     1.01s 22.70% 22.70%      1.17s 26.29%  compress/flate.(*compressor).findMatch
     0.66s 14.83% 37.53%      0.66s 14.83%  internal/filters.BoxBlur.func2
     0.58s 13.03% 50.56%      0.64s 14.38%  internal/filters.BoxBlur.func1
     0.19s  4.27% 54.83%      0.29s  6.52%  image/png.filter
     0.16s  3.60% 58.43%      0.16s  3.60%  runtime.memclrNoHeapPointers
     0.15s  3.37% 61.80%      0.16s  3.60%  compress/flate.matchLen (inline)
     0.13s  2.92% 64.72%      0.13s  2.92%  image/internal/imageutil.DrawYCbCr
     0.13s  2.92% 67.64%      0.15s  3.37%  image/jpeg.(*decoder).receiveExtend
     0.11s  2.47% 70.11%      0.18s  4.04%  image/jpeg.(*decoder).decodeHuffman
     0.11s  2.47% 72.58%      0.17s  3.82%  image/jpeg.(*decoder).reconstructBlock
     0.09s  2.02% 74.61%      0.31s  6.97%  image/png.(*decoder).readImagePass
     0.08s  1.80% 76.40%      0.11s  2.47%  internal/filters.Grayscale.func1
     0.07s  1.57% 77.98%      0.10s  2.25%  compress/flate.(*decompressor).huffSym
     0.07s  1.57% 79.55%      0.08s  1.80%  image/jpeg.(*decoder).ensureNBits
     0.06s  1.35% 80.90%      1.28s 28.76%  compress/flate.(*compressor).deflate
```

`BoxBlur.func1` y `func2` son las pasadas horizontal y vertical del desenfoque. El grafo completo está en `results/cpu_graph.svg`.

Anexo D · Reproducción y evidencias para la rúbrica

```
# compilar, generar datos y ejecutar (< 5 min, sin Internet: errgroup está en vendor/)
go build -o bin/pixelflow ./cmd/pixelflow && go build -o bin/genimages ./cmd/genimages
./bin/genimages -out data/in -n 120 -w 800 -h 600
./bin/pixelflow -in data/in -out data/out -filtros gray,blur:2,resize:50% -workers 4

# calidad y corrección
go vet ./... && gofmt -l . | grep -v ^vendor/
go test -race -count=3 ./...
go test -cover ./...

# demostraciones de la defensa
# primer error cancela todo (imagen 5 corrupta)
./bin/genimages -out data/bad -n 40 -corrupta 5 && ./bin/pixelflow -in data/bad -out data/o -q
# pulsar Ctrl+C a mitad → resumen parcial, código 130
./bin/pixelflow -in data/in -out data/o -q
# carrera introducida a propósito y detectada por -race
go test -race -tags racedemo -run RaceDemo ./internal/pipeline

# medición rápida desde la CLI y experimentos completos (gráficas: python3 scripts/plots.py)
./bin/pixelflow -in data/in -medir 3
./scripts/bench.sh 5

# interfaz web (Windows: go build -o bin\pixelflow-web.exe .\cmd\pixelflow-web)
go build -o bin/pixelflow-web ./cmd/pixelflow-web && ./bin/pixelflow-web   # http://localhost:8080
```

Criterio (peso)	Evidencia en la entrega
1 · Funcionalidad (20)	Tres filtros + Sobel; etapas listar→decodificar→filtrar→codificar configurables; pruebas píxel a píxel; salida idéntica byte a byte a `RunSequential`.
2 · Diseño concurrente (15)	Figura 1; fan-out configurable por etapa; un único responsable de cierre por canal; canales direccionales en las firmas; búferes acotados con cota medida.
3 · Corrección (15)	`-race -count=3` limpio; sin cierre múltiple posible; resultados idénticos con W = 1, 2, 4, 8; invariantes «exactamente una vez» y cota de memoria probados.
4 · Cancelación y fugas (10)	`errgroup` + `ctx` de extremo a extremo; Ctrl+C real en ≈81 ms; `checkNoLeaks` con `NumGoroutine`; ningún `time.Sleep`.
5 · Pruebas (10)	41 pruebas + 24 subpruebas: unitarias, integración (CLI y HTTP), adaptadores en memoria, error, concurrencia y cancelación; cobertura 87–99 %; sin Internet.
6 · Rendimiento (10)	GOMAXPROCS 1, 2, 4, 8 × 5 repeticiones con σ y CV; Amdahl; perfil pprof interpretado; cuello de botella identificado (codificación PNG).
7 · Código y repositorio (5)	`cmd/` + `internal/` en capas con puertos y adaptadores (Anexo E); 4 ADR; CI con gofmt, vet y `-race`; errores tipados y envueltos con `%w`; README reproducible; Makefile.
Extensiones (+)	Paralelismo de datos por franjas (resultado idéntico para cualquier k, probado), filtro Sobel e interfaz web con medición de speedup en vivo (Anexo F).

Anexo E · Arquitectura de software

Capas con puertos y adaptadores. Las flechas indican «depende de»; ningún paquete importa a uno de una capa exterior.

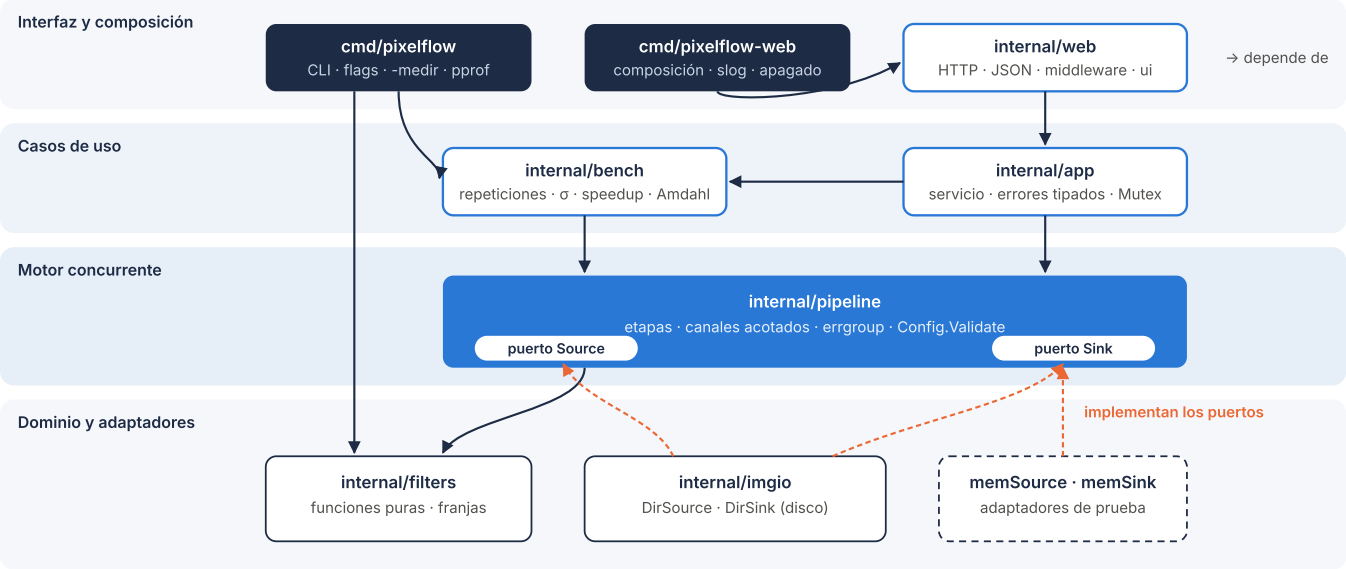


Figura E1. Paquetes y dependencias. `imgio` y los adaptadores de prueba implementan los puertos del pipeline sin importarlo (interfaces implícitas de Go).

Paquete	Responsabilidad	No sabe de
`filters`	Filtros puros sobre `*image.RGBA` y franjas paralelas.	Archivos, pipeline, HTTP
`pipeline`	Etapas, canales acotados, `errgroup`, métricas; puertos `Source` / `Sink`; única validación (`Config.Validate`).	Disco, HTTP
`imgio`	Adaptadores `DirSource` / `DirSink`; escritura atómica.	`pipeline` (no lo importa)
`bench`	Repeticiones intercaladas, media y σ, speedup, eficiencia y ajuste de Amdahl. Lo usan la CLI (`-medir`) y la web.	HTTP, interfaz
`app`	Casos de uso de la interfaz: imágenes, ejecuciones, mediciones, estado; errores tipados `ErrInvalid`, `ErrBusy`, `ErrNotFound`.	`net/http`
`web`	Rutas, JSON, traducción de errores a 400/409/404; middleware de recuperación de pánicos, registro con `slog` y cabeceras de seguridad.	Reglas de negocio, disco
`cmd/*`	Leer flags, componer las capas, apagar en orden (Ctrl+C).	Lógica

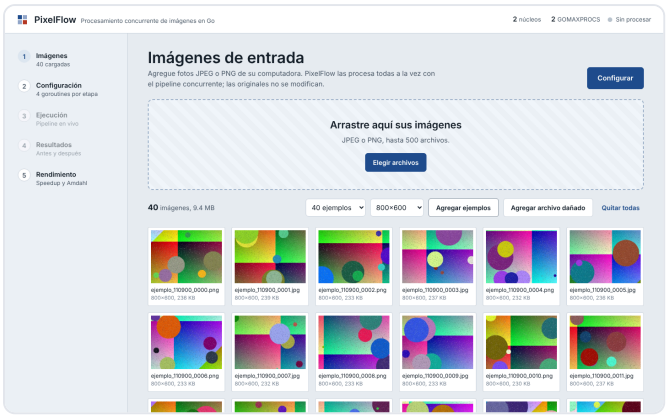
Decisiones registradas (docs/adr)

ADR	Decisión	Alternativa descartada
0001	Pipeline por etapas con canales acotados, closers con `WaitGroup` y `errgroup`.	Un solo pool que hace todo; cola con `Mutex` + `Cond`.
0002	Puertos `Source` y `Sink`; disco como adaptador.	Rutas fijas en `Config`; una interfaz `Storage` grande.
0003	Servicio (`app`) separado del adaptador HTTP (`web`).	Todo en `main`; framework web externo.
0004	Un trabajo a la vez, `Mutex` con instantáneas copiadas y consulta periódica.	WebSockets/SSE; `RWMutex`.

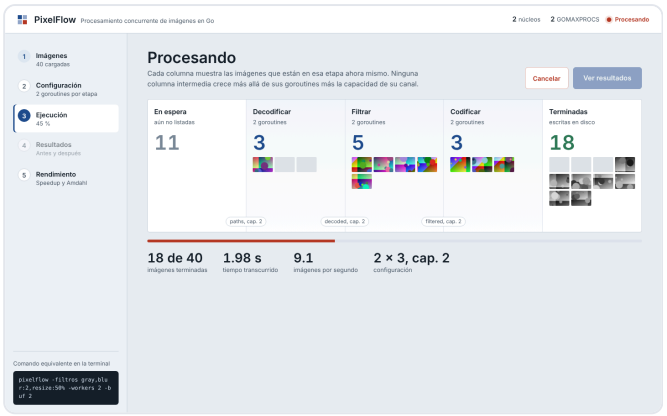
Integración continua en `.github/workflows/ci.yml`: `gofmt`, `go vet`, `go test -race -count=3` y cobertura en cada *push*.

Anexo F · Interfaz web (extensión)

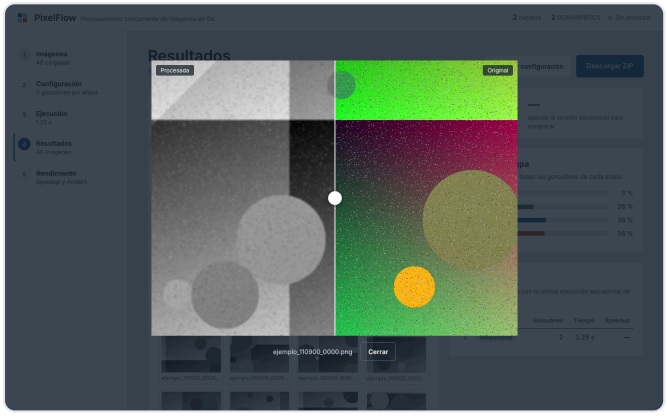
Flujo en cinco pasos sobre las mismas capas: imágenes, configuración, ejecución en vivo, resultados y rendimiento.



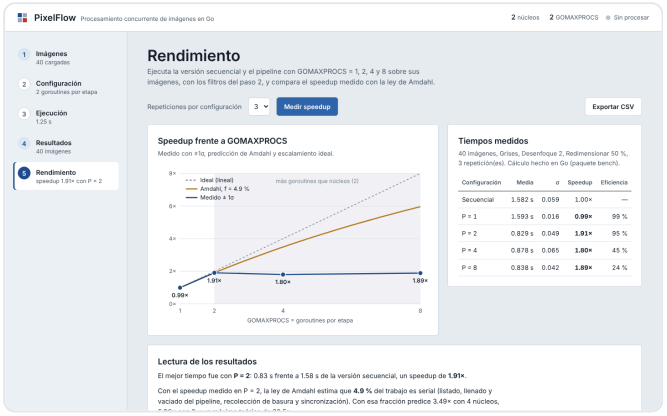
1 · Imágenes. Carga de fotos propias (JPEG/PNG validadas), ejemplos sintéticos y archivo dañado para la demostración de `errgroup`.



3 · Ejecución. Carriles en vivo alimentados por `OnStage`: las columnas intermedias nunca superan goroutines + capacidad del canal (contrapresión visible).



4 · Resultados. Comparador antes/después, tiempo por etapa, historial con speedup y descarga en ZIP.



5 · Rendimiento. Secuencial y GOMAXPROCS = 1, 2, 4, 8 con repeticiones; speedup frente a Amdahl calculado en Go (`bench`).